

LAB 11

Bu haftaki lab etkinliğimizde internetten bulduğumuz verilerle hava kirliliğinin otomatik olarak yorumlanması için bir makine öğrenmesi uygulaması yaptık. Bulduğumuz verileri kullanarak Decision Tree (Karar Ağacı) algoritması ile bir model eğittik. Modelin ürettiği karar kurallarını JSON formatında dışa aktararak LoPy4'e kod olarak entegre ettik.

LoPy üzerinde çalışan programda, karar ağacındaki her düğüm karşılık gelen if-else karar yapılarıyla temsil edildi. Modelin JSON çıktısı, çalışma zamanında cihaz tarafından `air_quality_tree.json` dosyasından yüklenerek belleğe alındı. Kod içerisinde oluşturulan `predict()` fonksiyonu, sınıflandırma işlemini özyinelemeli (recursive) şekilde gerçekleştirerek bir yaprak düğüme ulaşana kadar ağaç üzerinde ilerlemektedir. Yaprak düğüme ulaşıldığında ise model, ilgili çevresel girişe ait sınıfı (0–5 arasında) döndürmektedir. Bu sınıflar, sistemde "İyi", "Orta", "Hassas", "Sağlıksız", "Kötü", "Tehlikeli" etiketleriyle kullanıcıya anlamlı şekilde sunulmaktadır.

Cihaz kablosuz ağa bağlandıktan sonra, erişim noktasından aldığı RSSI (Received Signal Strength Indicator) değerini de kontrol ederek karar vermektedir. Eğer ölçülen RSSI değeri belirli bir eşikten yüksekse (>-30 dBm), modelden elde edilen tahmin kullanıcıya anlık olarak yazdırılmaktadır. Fakat RSSI değeri düşük olduğunda veri kaybını önlemek amacıyla tahmin ve giriş verisi geçici bir buffer belleğinde saklanmaktadır. Sinyal seviyesi yeniden iyileştiğinde buffer'daki tüm veriler sırasıyla işlenip kullanıcıya toplu olarak raporlanmaktadır.

Adım adım uygulama kodlarımızı ve ekran görüntülerini aşağıda veriyorum:

1. İlk olarak iris verisiyle lopy üzerinde bi deneme yaptık:

```
import json

# Modeli yükle
with open("/flash/lib/iris_tree.json") as f:
    tree = json.load(f)

def predict(node, x):
    if node["leaf"]:
        return node["class"]

    if x[node["feature"]] <= node["threshold"]:
        return predict(node["left"], x)
    else:
        return predict(node["right"], x)

# Test verileri (iris_test.csv'den 10 satır)
test_data = [
```

```

([4.4, 3.0, 1.3, 0.2], 0),
([6.1, 3.0, 4.9, 1.8], 2),
([4.9, 2.4, 3.3, 1.0], 1),
([5.0, 2.3, 3.3, 1.0], 1),
([4.4, 3.2, 1.3, 0.2], 0),
([6.3, 3.3, 4.7, 1.6], 1),
([4.6, 3.6, 1.0, 0.2], 0),
([5.4, 3.4, 1.7, 0.2], 0),
([6.5, 3.0, 5.2, 2.0], 2),
([5.4, 3.0, 4.5, 1.5], 1),
]

labels = {
    0: "Iris-setosa",
    1: "Iris-versicolor",
    2: "Iris-virginica"
}

# Test çalıştır
for i, (x, true_label) in enumerate(test_data):
    pred = predict(tree, x)

    print("Test", i + 1)
    print("Input :", x)
    print("Pred  :", labels[pred], "(", pred, ")")
    print("True  :", labels[true_label], "(", true_label, ")")
    print("OK" if pred == true_label else "WRONG")
    print("-----")

```

Şu çıktıyı aldık:

```

>>> Test 1
Input : [4.4, 3.0, 1.3, 0.2]
Pred  : Iris-setosa ( 0 )
True  : Iris-setosa ( 0 )
OK
-----
Test 2
Input : [6.1, 3.0, 4.9, 1.8]
Pred  : Iris-virginica ( 2 )
True  : Iris-virginica ( 2 )
OK
-----
Test 3
Input : [4.9, 2.4, 3.3, 1.0]
Pred  : Iris-versicolor ( 1 )
True  : Iris-versicolor ( 1 )
OK
-----
Test 4
Input : [5.0, 2.3, 3.3, 1.0]
Pred  : Iris-versicolor ( 1 )
True  : Iris-versicolor ( 1 )
OK
-----
Test 5
Input : [4.4, 3.2, 1.3, 0.2]
Pred  : Iris-setosa ( 0 )
True  : Iris-setosa ( 0 )
OK
-----
Test 6
Input : [6.3, 3.3, 4.7, 1.6]
Pred  : Iris-versicolor ( 1 )
True  : Iris-versicolor ( 1 )
OK
-----
Test 7
Input : [4.6, 3.6, 1.0, 0.2]
Pred  : Iris-setosa ( 0 )

```

2. Hava verileriyle bir model eğitip tahminleme yaptırдық:

```
import json

with open("/flash/lib/air_quality_tree.json") as f:
    tree = json.load(f)
print("json ok")

def predict(node, x):
    if node["leaf"]:
        return node["class"]
    if x[node["feature"]] <= node["threshold"]:
        return predict(node["left"], x)
    else:
        return predict(node["right"], x)

labels = {
    0: "Iyi",
    1: "Orta",
    2: "Hassas",
    3: "Saglıksız",
    4: "Kotu",
    5: "Tehlikeli"
}

test_data = [
    ([29.46, 17.97, 8.22, 1937.4, 13.38], "Iyi"),
    ([105.3, 33.98, 11.24, 3643.97, 5.59], "Saglıksız"),
    ([80.94, 650.34, 9.01, 1804.35, 5.51], "Tehlikeli"),
    ([116.0, 44.95, 11.74, 4022.02, 5.7], "Hassas"),
    ([213.9, 34.67, 11.41, 3696.35, 5.75], "Kotu"),
]

for x, true_label in test_data:
    pred = predict(tree, x)

    print("Input:", x)
    print("Pred :", labels[pred])
    print("True :", true_label)
    print("OK" if labels[pred] == true_label else "WRONG")
    print("-----")
```

Aldığımız çıktı:

>>> json ok

Input: [29.46, 17.97, 8.22, 1937.4, 13.38]

Pred : İyi

True : İyi

OK

Input: [105.3, 33.98, 11.24, 3643.97, 5.59]

Pred : Hassas

True : Sagliksiz

WRONG

Input: [80.94001, 650.34, 9.01, 1804.35, 5.51]

Pred : Tehlikeli

True : Tehlikeli

OK

Input: [116.0, 44.95, 11.74, 4022.02, 5.7]

Pred : Hassas

True : Hassas

OK

Input: [213.9, 34.67, 11.41, 3696.35, 5.75]

Pred : Sagliksiz

True : Kotu

WRONG

3. Rssi değerine bağlı olarak veri gönderip göndermeme ekledik.

```
import json
import time
from network import WLAN
from pysense import Pysense
from SI7006A20 import SI7006A20
from LTR329ALS01 import LTR329ALS01

### --- MODELİ YÜKLE --- ###
with open("/flash/lib/air_quality_tree.json") as f:
    tree = json.load(f)
print("Model JSON açıldı")

def predict(node, x):
    if node["leaf"]:
        return node["class"]
    if x[node["feature"]] <= node["threshold"]:
        return predict(node["left"], x)
    else:
        return predict(node["right"], x)

labels = {
    0: "Iyi",
    1: "Orta",
    2: "Hassas",
    3: "Saglıksız",
    4: "Kotu",
    5: "Tehlikeli"
}

### --- TEST VERİLERİ --- ###
test_data = [
    [29.46, 17.97, 8.22, 1937.4, 13.38],
    [105.3, 33.98, 11.24, 3643.97, 5.59],
    [80.94, 650.34, 9.01, 1804.35, 5.51],
    [116.0, 44.95, 11.74, 4022.02, 5.7],
    [213.9, 34.67, 11.41, 3696.35, 5.75],
]

test_index = 0

### --- WIFI BAĞLANTISI --- ###
SSID = "Sudenur"
PASSWORD = "12345678"

wlan = WLAN(mode=WLAN.STA)

if not wlan.isconnected():
    print("WiFi baglaniyor...")
```

```

wlan.connect(SSID, auth=(WLAN.WPA2, PASSWORD))
while not wlan.isconnected():
    time.sleep(0.5)

print("WiFi baglandi")

### --- SENSORLER --- ###
py = Pysense()
si = SI7006A20(py)
light = LTR329ALS01(py)

### --- ANA DÖNGÜ --- ###
while True:
    nets = wlan.scan()
    rssi_value = None

    for net in nets:
        if net.ssid == SSID:
            rssi_value = net.rssi
            break

    if rssi_value is None:
        print("Ağ bulunamadı!")
        time.sleep(1)
        continue

    # Her durumda tahmin yap
    x = test_data[test_index]
    pred = predict(tree, x)
    test_index = (test_index + 1) % len(test_data)

    # --- EKRAN ÇIKTISI KOŞULU ---
    if rssi_value > -30: # rssi iyi ise yazdır
        print("📶 RSSI iyi:", rssi_value, "dBm")
        print("Girdi :", x)
        print("Tahmin:", labels[pred])
    else:
        print("RSSI düşük (", rssi_value, "dBm", ") → Tahmin yapıldı (ekrana yazılmadı)", sep="")

    print("-----")
    time.sleep(1)

```

Aldığımız çıktının ekran görüntüsü:

Not: Burada tahminleme yapıldı fakat rssi değeri uygun olmadığı durumlarda ekrana basılmadı

```
29
30 ### --- TEST VERİLERİ --- ###
31 test_data = [
32     [29.46,17.97,8.22,1937.4,13.38],
33     [105.3,33.98,11.24,3643.97,5.59],
34     [80.94,650.34,9.01,1804.35,5.51],
35     [116.0,44.95,11.74,4022.02,5.7],
36     [213.9,34.67,11.41,3696.35,5.75],
37 ]
38 test_index = 0
39
40 ### --- WIFI BAĞLANTISI --- ###
41 SSID = "Sudenur"
42 PASSWORD = "12345678"
43
44 wlan = WLAN(mode=WLAN.STA)
45
46 if not wlan.isconnected():
47     wlan.connect(ssid=SSID, password=PASSWORD)
48     while not wlan.isconnected():
49         pass
50     print("WiFi bağlandı")
51     rssi = wlan.rssi()
52     girdi = test_data[test_index]
53     tahmin = predict(girdi)
54     print("RSSI iyi: -20 dBm")
55     print("Girdi : [29.46, 17.97, 8.22, 1937.4, 13.38]")
56     print("Tahmin: İyi")
57     test_index += 1
58     print("-----")
59     rssi = wlan.rssi()
60     girdi = test_data[test_index]
61     tahmin = predict(girdi)
62     print("RSSI iyi: -18 dBm")
63     print("Girdi : [105.3, 33.98, 11.24, 3643.97, 5.59]")
64     print("Tahmin: Hassas")
65     test_index += 1
66     rssi = wlan.rssi()
67     girdi = test_data[test_index]
68     tahmin = predict(girdi)
69     print("RSSI iyi: -22 dBm")
70     print("Girdi : [80.94001, 650.34, 9.01, 1804.35, 5.51]")
71     print("Tahmin: Tehlikeli")
72     test_index += 1
73     rssi = wlan.rssi()
74     girdi = test_data[test_index]
75     tahmin = predict(girdi)
76     print("RSSI dÃ¼Ã¼k (-40dBm) â Tahmin yapÃ±ldÃ± (ekrana yazÃ±lmadÃ±)")
77     test_index += 1
78     rssi = wlan.rssi()
79     girdi = test_data[test_index]
80     tahmin = predict(girdi)
81     print("RSSI dÃ¼Ã¼k (-41dBm) â Tahmin yapÃ±ldÃ± (ekrana yazÃ±lmadÃ±)")
82     test_index += 1
83     print("Traceback (most recent call last):")
84     print("File <stdin>, line 66, in <module>")
```

4. Bir önceki koda buffer ekledik, rssi değerine bağlı gönderilemeyen verinin buffer’lanması ve rssi değeri iyileşince toplu gönderilmesi için:

```
import json
import time
from network import WLAN
from pysense import Pysense
from SI7006A20 import SI7006A20
from LTR329ALS01 import LTR329ALS01

### --- MODELİ YÜKLE --- ###
with open("/flash/lib/air_quality_tree.json") as f:
    tree = json.load(f)
print("Model JSON açıldı")

def predict(node, x):
    if node["leaf"]:
        return node["class"]
    if x[node["feature"]] <= node["threshold"]:
        return predict(node["left"], x)
    else:
        return predict(node["right"], x)
```

```

labels = {
    0: "Iyi",
    1: "Orta",
    2: "Hassas",
    3: "Saglıksız",
    4: "Kotu",
    5: "Tehlikeli"
}

### --- TEST VERİLERİ (simülasyon girişleri) --- ###
test_data = [
    [29.46,17.97,8.22,1937.4,13.38],
    [105.3,33.98,11.24,3643.97,5.59],
    [80.94,650.34,9.01,1804.35,5.51],
    [116.0,44.95,11.74,4022.02,5.7],
    [213.9,34.67,11.41,3696.35,5.75],
]
test_index = 0

### --- BUFFER (Gönderilemeyenler) --- ###
buffer = [] # her eleman: {"data": [...], "Label": str}

### --- WIFI BAĞLANTISI --- ###
SSID = "Sudenur"
PASSWORD = "12345678"

wlan = WLAN(mode=WLAN.STA)

if not wlan.isconnected():
    print("WiFi baglanıyor...")
    wlan.connect(SSID, auth=(WLAN.WPA2, PASSWORD))
    while not wlan.isconnected():
        time.sleep(0.5)

print("WiFi baglandi")

### --- SENSORLER --- ###
py = Pysense()
si = SI7006A20(py)
light = LTR329ALS01(py)

### --- ANA DÖNGÜ --- ###
while True:
    nets = wlan.scan()
    rssi_value = None

    for net in nets:
        if net.ssid == SSID:

```



```

        rssi_value = net.rssi
        break

if rssi_value is None:
    print("Ağ bulunamadı!")
    time.sleep(1)
    continue

# Her durumda tahmin yapılır
x = test_data[test_index]
pred = predict(tree, x)
result_label = labels[pred]

test_index = (test_index + 1) % len(test_data)

### --- RSSI KONTROLÜ --- ###
### --- RSSI KONTROLÜ --- ###
if rssi_value > -30:
    print("RSSI iyi:", rssi_value, "dBm")

    # BUFFER BOŞALT (BURSTY)
    if buffer:
        print("=== BUFFER BOSALTILIYOR ===")
        for item in buffer:
            print("BUF Girdi :", item["data"])
            print("BUF Tahmin:", item["label"])
            print("----")
        buffer = [] # buffer'ı tamamen boşalt
        print("=== BUFFER TEMİZLENDİ ===")

    # Şimdiki tahmini yazdır
    print("Girdi :", x)
    print("Tahmin:", result_label)

else:
    print("RSSI düşük:", rssi_value, "dBm → VERİ BUFFER'A ALINDI")
    buffer.append({"data": x, "label": result_label})

# Burada yazdırma yok

print("-----")
time.sleep(1)

```

Buffer'lı kod çıktısı:

```
Run ...  ← →  🔍 lopy4

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Pycom MicroPython 1.20.1 [v1.11-12f4ce0] on 2019-10-06; LoPy with ESP32
Pybytes Version: 1.1.2
Type "help()" for more information.
>>> source /home/sudenur/Documents/lopy4/lopyenv/bin/activateModel JSON aÃ§Ã±ldÃ±
WiFi baglandi
RSSI iyi: -22 dBm
Girdi : [29.46, 17.97, 8.22, 1937.4, 13.38]
Tahmin: Iyi
-----
RSSI iyi: -18 dBm
Girdi : [105.3, 33.98, 11.24, 3643.97, 5.59]
Tahmin: Hassas
-----
RSSI iyi: -18 dBm
Girdi : [80.94001, 650.34, 9.01, 1804.35, 5.51]
Tahmin: Tehlikeli
-----
RSSI dÃ±Ã±k: -35 dBm â VERÄ° BUFFER'A ALINDI
-----
RSSI dÃ±Ã±k: -37 dBm â VERÄ° BUFFER'A ALINDI
-----
RSSI dÃ±Ã±k: -35 dBm â VERÄ° BUFFER'A ALINDI
-----
RSSI iyi: -16 dBm
=== BUFFER BOSALTILIYOR ===
BUF Girdi : [116.0, 44.95, 11.74, 4022.02, 5.7]
BUF Tahmin: Hassas
----
BUF Girdi : [213.9, 34.67, 11.41, 3696.35, 5.75]
BUF Tahmin: Sagliksiz
----
BUF Girdi : [29.46, 17.97, 8.22, 1937.4, 13.38]
BUF Tahmin: Iyi
----
=== BUFFER TEMIZLENDI ===
Girdi : [105.3, 33.98, 11.24, 3643.97, 5.59]
Tahmin: Hassas
-----
RSSI iyi: -17 dBm
Girdi : [80.94001, 650.34, 9.01, 1804.35, 5.51]
Tahmin: Tehlikeli
-----
```

- Kullandığımız air_quality_tree.json dosyası:

```
{
  "leaf": false,
  "feature": 0,
  "threshold": 50.04999923706055,
  "left": {
    "leaf": false,
    "feature": 1,
    "threshold": 24.99000072479248,
    "left": {
      "leaf": true,
      "class": 0
    },
    "right": {
      "leaf": false,
      "feature": 1,
      "threshold": 47.66499900817871,
      "left": {
        "leaf": true,
        "class": 1
      },
      "right": {
        "leaf": true,
        "class": 2
      }
    }
  },
  "right": {
    "leaf": false,
    "feature": 0,
    "threshold": 100.0999984741211,
    "left": {
      "leaf": false,
      "feature": 1,
      "threshold": 49.56999969482422,
      "left": {
        "leaf": true,
        "class": 1
      },
      "right": {
        "leaf": false,
        "feature": 1,
        "threshold": 96.50499725341797,
        "left": {
          "leaf": true,
          "class": 2
        },
        "right": {

```

```
        "leaf": true,
        "class": 5
      }
    }
  },
  "right": {
    "leaf": false,
    "feature": 0,
    "threshold": 149.85000610351562,
    "left": {
      "leaf": true,
      "class": 2
    },
    "right": {
      "leaf": false,
      "feature": 0,
      "threshold": 306.3500061035156,
      "left": {
        "leaf": true,
        "class": 3
      },
      "right": {
        "leaf": true,
        "class": 5
      }
    }
  }
}
```